

دیتابیس

- نوع دیتابیس MySQL :
- دلیل انتخاب: قابلیت مدیریت مناسب Lock و Concurrency در سطح بالا.
- برخلاف دیتابیس پیش فرض (SQLite) Django ، MySQL قابلیت Row-level Locking را به خوبی پشتیبانی می کند.
- اطلاعات اتصال به دیتابیس از فایل env موجود در root پروژه خوانده می شود.
- در صورت وصل نشدن کانکشن دیتابیس اپلیکیشن متوقف می شود

مدل ها

پروژه دارای دو مدل اصلی Wallet و TransactionLog است.

به دلیل نیاز به موجودیت های User و Symbol، این دو مدل نیز به صورت ساده پیاده سازی شده اند. با توجه به تمرکز اصلی پروژه بر روی والت و تراکنش ها، بخش های پیشرفته مربوط به User مانند Authentication کامل پیاده سازی نشده است. در حال حاضر، در هر درخواست user_id به همراه داده ارسال می شود.

مثال

http://127.0.0.1:8000/api/wallet/withdraw/

```
{
  "wallet_id": "22f48518-8c99-4524-8a45-76b8ec8eae8a",
  "user_id": "c647929a-8ba3-4ace-a7a4-1c4927df5c28",
  "amount": "50"
  "refrence_key": "testKey"
}
```

بررسی دقیق مدل ها

1. Wallet

- wallet_id

برای قابل حدس نبودن از نوع UUID است و unique می باشد.

- دارای دو ForeignKey

به مدل های User و Symbol

Symbol معنی نوع دارایی است که داخل والت ذخیره شده برای مثال BTC ، این کار قابلیت افزودن چند نوع والت برای هر کاربر اما با ارز متفاوت را به ما می دهد.

برای هندل کردن این موضوع، در مدل باید (user, symbol) باهم منحصر به فرد باشند.

- balance
- از نوع Decimal با max_digits=32 و decimal_places=16.
- دارای دو فیلد created_at و updated_at برای ذخیره کردن زمان.
- 2. TransactionLog
 - transaction_id
 - برای قابل حدس نبودن از نوع UUID است و unique می‌باشد.
 - دارای یک ForeignKey به مدل Wallet
 - برای ذخیره‌سازی اطلاعات والت. با توجه به نیاز به سیو کردن همه تراکنش‌ها، on_delete=models.PROTECT تنظیم شده است.
 - transaction_type
 - از نوع Enum با مقادیر DEPOSIT و WITHDRAW
 - amount: با مشخصاتی شبیه به balance در مدل Wallet.
 - دارای فیلد created_at برای ذخیره‌سازی زمان.
 - هر تراکنش دارای یک reference_key منحصر به فرد می‌باشد که از سمت فرانت ارسال می‌شود.
 - مکانیزم reference_key
 - جلوگیری از ثبت دوباره تراکنش: این قسمت برای جلوگیری از ثبت دوباره تراکنش در صورت Retry اضافه شده است. با آمدن هر درخواست، ابتدا چک می‌شود که reference_key در جدول TransactionLog وجود دارد یا خیر:
 - اگر موجود بود، به عنوان Duplicate شناخته شده و داده از قبل ذخیره شده به عنوان ریسپانس با status 200 ارسال می‌شود.
 - اگر وجود نداشت، درخواست انجام شده و نتیجه آن در جدول ذخیره می‌گردد.
- 3. User
 - user_id
 - name
 - created_at
- 4. Symbol
 - symbol_id
 - name
 - created_at

Error handling and validation

برای بررسی دیتای ریکوئست های مختلف و همچنین ارور های مرتبط به دیتابیس custom exeptions طراحی شده که در صورت رخ دادن raise شده و پیام مورد نظر را در ریسپانس به صورت زیر نمایش میدهد

```
{
  "success" : false,
  "message" : "custom message"
}
```

ولیدیشن در Serializers

User و Symbol Serializers

- فیلد name باید حداقل ۲ کاراکتر باشد.
- نام فقط می تواند شامل حروف و اعداد باشد (بدون کاراکتر خاص).

Wallet Serializer

- symbol_name از نوع CharField و باید حتماً از قبل در جدول Symbol وجود داشته باشد.

- user_id

از نوع UUIDField.

Transaction Serializer

- wallet_id

از نوع UUIDField و بررسی می شود که حتماً در جدول Wallet وجود داشته باشد.

- amount

amount = serializers.DecimalField(max_digits=18, decimal_places=16)

مقدار آن باید مثبت باشد.

- user_id

از نوع UUIDField

- reference_key

از نوع CharField.

در صورت عدم رعایت هر یک از شرایط فوق، با استفاده از serializers.ValidationError و custom message خطا raise می‌شود و در ریسپانس به کاربر نمایش داده خواهد شد.

transaction_service functions

Error handling in except	Flow	Params	Name
except Wallet.DoesNotExist : raise WalletUnauthorizedError(walletUnauthorizedError) except IntegrityError: raise DuplicateTransactionError(transactionLogDuplicatedError) except OperationalError: raise TransactionDatabaseUnavailableError(databaseUnavaiaibleError) except DatabaseError : raise TransactionDatabaseError(databaseError)	کل فانکشن توسط بلوک try و except احاطه شده است. در صورت رخ دادن هرگونه خطا، یک Custom Exception raise می‌شود. در ابتدای تابع، ابتدا بررسی می‌شود که آیا reference_key قبلاً در جدول TransactionLog وجود دارد یا خیر. اگر وجود داشت، داده‌های تراکنش ذخیره شده قبلی بلافاصله بازگشت داده می‌شود. در غیر این صورت، عملیات اصلی داخل تراکنش اتمیک (transaction.atomic) انجام می‌شود. ابتدا والت با استفاده از select_for_update و wallet_id و user_id انتخاب و لاک می‌شود. سپس مقدار به بالانس والت اضافه یا کسر شده و ذخیره می‌گردد. در نهایت، رکورد جدید در جدول TransactionLog ذخیره می‌شود.	wallet_id amount user_id refrence_key	deposit
except Wallet.DoesNotExist : raise WalletUnauthorizedError(walletUnauthorizedError) except IntegrityError: raise DuplicateTransactionError(transactionLogDuplicatedErro) except OperationalError: raise TransactionDatabaseUnavailableError(databaseUnavaiaibleError) except DatabaseError : raise TransactionDatabaseError(databaseError)	این فانکشن مشابه فانکشن قبلی عمل می‌کند با این تفاوت که قبل از کم کردن مقدار از بالانس والت، شرط زیر چک می‌شود: اگر موجودی والت کمتر از مقدار درخواستی باشد، یک Custom Exception به نام WithdrawLargeAmount raise می‌شود. در صورت کافی بودن موجودی، عملیات کسر از بالانس انجام شده و ادامه فرآیند (ذخیره تراکنش در TransactionLog) اجرا می‌گردد.	wallet_id amount user_id refrence_key	withdraw

<pre>except Wallet.DoesNotExist : raise WalletUnauthorizedError(walletUnauthorizedError) except OperationalError: raise TransactionDatabaseUnavailableError(databaseUnavaiaibleError) except DatabaseError : raise TransactionDatabaseError(databaseError)</pre>	این فانکشن داخل بلوک try و except قرار دارد. در این تابع، والت مورد نظر بر اساس wallet_id و user_id خوانده می‌شود، سپس مقدار balance از آن استخراج شده و به عنوان پاسخ به کاربر برگردانده می‌شود.	wallet_id user_id	get_balance
<pre>except OperationalError: raise TransactionDatabaseUnavailableError(databaseUnavaiaibleError) except DatabaseError: raise TransactionDatabaseError(databaseError)</pre>	این فانکشن داخل بلوک try و except قرار دارد. ابتدا یک کوئری از جدول TransactionLog با استفاده از select_related اطلاعات مربوط به والت و کاربر نیز به صورت یکجا لود می‌گردد. اگر wallet_id ارسال شده باشد، تراکنش‌ها فقط برای آن والت خاص فیلتر می‌شوند. اگر user_id ارسال شده باشد، تراکنش‌های مربوط به تمام والت‌های آن کاربر فیلتر می‌شوند. در نهایت، تراکنش‌ها بر اساس created_at به صورت نزولی (جدیدترین اول) مرتب شده و بازگشت داده می‌شوند.	wallet_id=None user_id=None	get_transactions

Api list

-----WALLET-----

get all wallets

GET

<http://127.0.0.1:8000/api/wallets/>

Response :

```
{
  "success": true,
  "data": [
    {
      "wallet_id": "d3b92529-ea33-451f-af17-2a369105e157",
      "balance": "0.00",
```

```
    "user_id": "5a1958c0-4ce8-40aa-939a-6df6ecfb65d0"
  }
]
}
```

create wallet

POST

<http://127.0.0.1:8000/api/wallet/>

Request :

```
{
  "symbol_name": "BTC",
  "user_id": "c647929a-8ba3-4ace-a7a4-1c4927df5c28"
}
```

Response :

```
{
  "success": true,
  "message": "Wallet created",
  "data": {
    "walletId": "22f48518-8c99-4524-8a45-76b8ec8eae8a"
  }
}
```

deposit

POST

<http://127.0.0.1:8000/api/wallet/deposit/>

Request :

```
{
  "wallet_id": "22f48518-8c99-4524-8a45-76b8ec8eae8a",
  "user_id": "c647929a-8ba3-4ace-a7a4-1c4927df5c28",
}
```

```
"amount": "50" ,  
"refrence_key" : "gcfghcfj2"  
}
```

Response :

```
{  
  "success": true,  
  "data": {  
    "walletId": "22f48518-8c99-4524-8a45-76b8ec8eae8a",  
    "balance": 50.0,  
    "transactionId": "1079caae-f2fc-484b-897d-6c05b414ea02"  
  }  
}
```

withdraw

POST

<http://127.0.0.1:8000/api/wallet/withdraw/>

Request :

```
{  
  "wallet_id" : "22f48518-8c99-4524-8a45-76b8ec8eae8a" ,  
  "user_id" : "c647929a-8ba3-4ace-a7a4-1c4927df5c28",  
  "amount": "50" ,  
  "refrence_key" : "gcfghcfj2"  
}
```

Response :

```
{  
  "success": true,  
  "data": {  
    "walletId": "22f48518-8c99-4524-8a45-76b8ec8eae8a",  
    "balance": 0.0,  
    "transactionId": "44195aa7-fc91-43a3-be70-fa989516d65f"
```

```
}  
}
```

get transaction history

GET

<http://127.0.0.1:8000/api/transactions/>

http://127.0.0.1:8000/api/transactions/?wallet_id=7b5ba0fb-b7c9-40cb-806f-5c8bc7faeac1

http://127.0.0.1:8000/api/transactions/?user_id=c647929a-8ba3-4ace-a7a4-1c4927df5c28

Response :

```
{  
  "success": true,  
  "data": [  
    {  
      "transactionId": "193bebcf-8bf8-4cdd-b482-1f3bc03f5920",  
      "walletId": "22f48518-8c99-4524-8a45-76b8ec8eae8a",  
      "userId": "c647929a-8ba3-4ace-a7a4-1c4927df5c28",  
      "amount": "50.00",  
      "transactionType": "WITHDRAW",  
      "created_at": "2026-05-23T08:39:37.800047Z"  
    },  
  ]  
}
```

get wallet balance

GET

http://localhost:8000/api/wallet/balance/?wallet_id=22f48518-8c99-4524-8a45-76b8ec8eae8a&user_id=c647929a-8ba3-4ace-a7a4-1c4927df5c28

Response :

```
{  
  "success": true,  
  "data": {
```



```
"balance": 0.0
}
}
```

----- USER AND SYMBOL -----

user and symbol

create user

POST

<http://localhost:8000/api/user/>

Request :

```
{
  "name" : "Bita"
}
```

Response :

```
{
  "success": true,
  "data": {
    "user_id": "68164ba2-9f2d-4a1a-b4e0-4100bf9342d5",
    "name": "Bita"
  }
}
```

get all users

GET

<http://localhost:8000/api/users/>

Response :

```
{
  "success": true,
  "data": [
    {
```

```
    "user_id": "5a1958c0-4ce8-40aa-939a-6df6ecfb65d0",  
    "name": "user1"  
  },  
]  
}
```

create symbol

POST

<http://127.0.0.1:8000/api/symbol/>

Request:

```
{  
  "name" : "BTC"  
}
```

get all symbols

GET

<http://127.0.0.1:8000/api/symbols/>

Response :

```
{  
  "success": true,  
  "data": [  
    {  
      "symbol_id": "9c91dfff-d009-4816-a680-49916b32b701",  
      "name": "BTC"  
    },  
  ]  
}
```

Test list

1. CreateWalletApiTest

- test_create_wallet_invalid_user_id
- test_create_wallet_invalid_symbol_name
- test_create_wallet_missing_symbol_name
- test_create_wallet_missing_user_id
- test_duplicated_wallet
- test_creaet_wallet_successful

2. DepositRaceConditionTest

- test_concurrent_deposits

3. AtomicRollbackApiTest

- test_atomic_deposit_api_rollback
- test_atomic_withdraw_api_rollback

4. AtomicRollbackTest

- test_atomic_deposit_rollback
- test_atomic_withdraw_rollback

5. GetBalanceApiTest

- test_get_balance
-

6. DepositApiTest

- test_deposit_one_time_success
- test_withdraw_one_time_success
- test_withdraw_larger_than_balance
- test_negetive_deposit
- test_retry_deposit

7. TransactionHistoryApiTest

- test_transaction_history_empty
- test_transaction_history_success

8. ThroughPutTests(The throughput tests are in the root of the project and require running the server first.)

- deposit_throughput_test

```
Database connected
Using database: wallet_db
Running 10000 deposit requests with 20 workers...

Wallet found | Starting Balance: 1000.0000000000000000
```

```
=====
THROUGHPUT TEST RESULTS (REAL DATABASE)
=====
```

```
Total Requests   : 10000
Successful        : 10000
Failed           : 0
Duration          : 458.62 seconds
Throughput        : 21.80 req/sec
Success Rate      : 100.0%
Balance Increase  : 10000.0000000000000000
=====
```

- `withdraw_throuput_test`

```
Wallet found | Starting Balance: 101000.0000000000000000
```

```
=====
THROUGHPUT TEST RESULTS (REAL DATABASE)
=====
```

```
Total Requests   : 10000
Successful        : 10000
Failed           : 0
Duration          : 442.88 seconds
Throughput        : 22.58 req/sec
Success Rate      : 100.0%
Balance Increase  : -100000.0000000000000000
=====
```